Laborator_02

Complete C# Fundamentals Study Guide

Table of Contents

1. C# Type System Overview
2. Namespaces
3. Classes
4. Structs (Structures)
5. Records
6. Interfaces
7. Generics
8. Anonymous Types
9. Strings

C# Type System Overview

What is a Type?

C# is a **strongly typed language**. Every variable, constant, and expression that evaluates to a value has a type. The compiler uses type information to ensure **type safety** in your code.

Key Concepts

Common Type System (CTS)

- All types in C# derive from `System.Object` (keyword: `object`)
- This creates a **unified type hierarchy**
- Even primitive types like `int` derive from `object`

Value Types vs Reference Types

Value Types:

- Store data directly
- Allocated on the stack (or inline in containing types)
- Copying a value type creates a new copy of the data
- Examples: `int`, `double`, `bool`, `struct`, `enum`

- Derive from `System.ValueType`

Reference Types:

- Store a reference to the data's location
- Allocated on the managed heap
- Copying a reference type copies the reference, not the data
- Multiple variables can reference the same object
- Examples: `class`, `interface`, `delegate`, `array`, `string`
- Derive from `System.Object`

Type Categories

Built-in Types

C# provides standard built-in types representing:

- Integers (`int`, `long`, `short`, `byte`)
- Floating point values (`float`, `double`)
- Boolean expressions (`bool`)
- Text characters (`char`)
- Decimal values (`decimal`)
- Strings (`string`)
- Objects (`object`)

Custom Types

You create custom types using:

- `struct` - Value type
- `class` - Reference type
- `interface` - Contract definition
- `enum` - Named constants
- `record` - Immutable data types
- `delegate` - Type-safe function pointers

Type Information Stored in Metadata

Each type contains:

- Storage space required
- Maximum and minimum values
- Members (methods, fields, events, properties, etc.)
- Base type it inherits from

- Interfaces it implements
- Permitted operations

Choosing the Right Type

Choose a struct when:

- Data storage is small (≤64 bytes)
- Type is immutable
- You want value semantics for equality
- Type primarily stores data, not behavior
- No need for inheritance

Choose a class when:

- Type uses polymorphism
- Type is part of an inheritance hierarchy
- Primary purpose is behavior
- Data will be modified after creation

Choose a record when:

- Type is immutable or you want nondestructive mutation
- You want value semantics for equality
- Type primarily stores data
- Content intended for use outside the conversation

Namespaces

What is a Namespace?

Namespaces are used to organize code and create globally unique types. They act as containers for classes, structs, interfaces, enums, delegates, and other namespaces.

Why Use Namespaces?

1. **Organization** - Group related code together
2. **Avoid naming conflicts** - Two classes can have the same name if in different namespaces
3. **Manage large projects** - Essential for large codebases

Declaring a Namespace

Traditional Syntax:

```
namespace MyCompany.MyProduct
{
    class MyClass
    {
        public void MyMethod()
        {
            System.Console.WriteLine("Hello from MyNamespace");
        }
    }
}
```

File-Scoped Namespace (C# 10+):

```
namespace MyCompany.MyProduct;

class MyClass
{
    public void MyMethod()
    {
        System.Console.WriteLine("Hello from MyNamespace");
    }
}
```

Using Namespaces

The `using` Directive:

```
using System;
using System.Collections.Generic;

namespace MyApp
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("No need for System.Console!");
            List<string> names = new List<string>(); // No need for full
name
        }
    }
}
```

Using Alias:

```
using Project = MyCompany.MyProduct.MyProject;

// Now use Project instead of full namespace
Project.MyClass obj = new Project.MyClass();
```

Global Using (C# 10+):

```
global using System;
global using System.Collections.Generic;

// These apply to all files in the project
```

Nested Namespaces

```
namespace Outer
{
    namespace Inner
    {
        class MyClass { }
    }
}

// Accessed as: Outer.Inner.MyClass
```

Namespace Naming Conventions

- Use PascalCase
- Format: `<Company>.<Product>.<Feature>.<Subnamespace>`
- Example: `Microsoft.Office.PowerPoint`
- Use plural names where appropriate: `System.Collections`
- Avoid conflicts with commonly used namespaces

The Global Namespace

- Unnamed, default namespace
- Present in every file
- Contains declarations not in any declared namespace

Classes

What is a Class?

A class is a **reference type** that can encapsulate data (fields, properties) and behavior (methods, events). It serves as a blueprint for creating objects.

Declaring a Class

```
public class Person
{
    // Fields (private data)
    private string name;
    private int age;

    // Properties (public access to data)
    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    public int Age
    {
        get { return age; }
        set { age = value; }
    }

    // Constructor
    public Person(string name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Method
    public void Introduce()
    {
        Console.WriteLine($"Hi, I'm {name} and I'm {age} years old.");
    }
}
```

Creating Objects (Instances)

```
// Create an instance
Person person1 = new Person("John", 30);
person1.Introduce();

// Multiple variables can reference the same object
Person person2 = person1;
person2.Name = "Jane";
```

```
Console.WriteLine(person1.Name); // Outputs: Jane (same object!)
```

Class Members

Classes can contain:

- **Fields** - Variables that store data
- **Properties** - Provide controlled access to fields
- **Methods** - Define behavior
- **Events** - Enable notifications
- **Constructors** - Initialize new instances
- **Finalizers/Destructors** - Cleanup before garbage collection
- **Indexers** - Allow array-like access
- **Operators** - Define custom operators
- **Nested Types** - Classes, structs, etc. inside the class

Access Modifiers

```
public class Example
{
    public int PublicField;        // Accessible anywhere
    private int PrivateField;      // Only within this class
    protected int ProtectedField;  // This class and derived classes
    internal int InternalField;    // Within same assembly
    protected internal int ProtectedInternalField; // Protected OR internal
    private protected int PrivateProtectedField;   // Protected AND internal
}
```

Default accessibility: `private`

Inheritance

```
// Base class
public class Animal
{
    public virtual void MakeSound()
    {
        Console.WriteLine("Some generic sound");
    }
}

// Derived class
public class Dog : Animal
{
    public override void MakeSound()
```

```
    {
        Console.WriteLine("Woof!");
    }
}
```

Important Notes:

- A class can only inherit from **ONE** base class
- A class can implement **MULTIPLE** interfaces
- Use `virtual` to allow methods to be overridden
- Use `override` to override virtual methods
- Use `sealed` to prevent further inheritance

Abstract Classes

```
public abstract class Shape
{
    // Abstract method - no implementation
    public abstract double CalculateArea();

    // Regular method with implementation
    public void Display()
    {
        Console.WriteLine($"Area: {CalculateArea()}");
    }
}

public class Circle : Shape
{
    public double Radius { get; set; }

    // Must implement abstract methods
    public override double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

// Cannot instantiate abstract class
// Shape shape = new Shape(); // ERROR!
Circle circle = new Circle { Radius = 5 };
```

Sealed Classes

```
public sealed class FinalClass
{
    // Cannot be inherited
```

```
}

// This will cause an error:
// public class DerivedClass : FinalClass { }
```

Static Classes

```
public static class MathHelper
{
    public static int Add(int a, int b)
    {
        return a + b;
    }
}

// Cannot instantiate static class
// MathHelper helper = new MathHelper(); // ERROR!

// Call static methods directly
int result = MathHelper.Add(5, 3);
```

Static class rules:

- Cannot be instantiated
- Contains only static members
- Cannot inherit from other classes
- Sealed by default

Partial Classes

```
// File1.cs
public partial class Employee
{
    public string Name { get; set; }
}

// File2.cs
public partial class Employee
{
    public int Id { get; set; }

    public void Display()
    {
        Console.WriteLine($"{Id}: {Name}");
    }
}
```

```
// Both parts combine into one class
```

Structs (Structures)

What is a Struct?

A struct is a **value type** that can encapsulate data and related functionality. Structs are similar to classes but have different behavior.

Declaring a Struct

```
public struct Point
{
    public double X { get; set; }
    public double Y { get; set; }

    // Constructor
    public Point(double x, double y)
    {
        X = x;
        Y = y;
    }

    // Method
    public double DistanceFromOrigin()
    {
        return Math.Sqrt(X * X + Y * Y);
    }
}
```

Value Type Behavior

```
Point p1 = new Point(10, 20);
Point p2 = p1; // Copies the value

p2.X = 30; // Only p2 is changed

Console.WriteLine(p1.X); // Still 10
Console.WriteLine(p2.X); // Now 30
```

Key Differences: Struct vs Class

Feature	Struct	Class
Type	Value type	Reference type
Memory	Stack (usually)	Heap
Copying	Copies data	Copies reference
Inheritance	Cannot inherit from structs	Can inherit from classes
Default constructor	Always provided	Can be customized
Finalize r	Not allowed	Allowed
`null` value	Nullable with `?`	Can be null by default

Readonly Struct

```
public readonly struct ImmutablePoint
{
    public double X { get; }
    public double Y { get; }

    public ImmutablePoint(double x, double y)
    {
        X = x;
        Y = y;
    }
}

// All members are automatically readonly
// Cannot modify after creation
```

Ref Struct

```
public ref struct RefStructExample
{
    private Span<int> data;

    public RefStructExample(Span<int> data)
    {
        this.data = data;
    }
}
```

Ref struct rules:

- Must be allocated on the stack
- Cannot escape to the managed heap

- Cannot be boxed
- Cannot be used in async methods
- Cannot be array element type
- Examples: `Span<T>`, `ReadOnlySpan<T>`

When to Use Structs

Use structs when:

- Type represents a single value
- Instance size is small (≤ 16 bytes recommended)
- Type is immutable
- Won't be boxed frequently
- Logically similar to primitive types (`int` , `double` , etc.)

Examples of good structs:

- `Point` , `Rectangle` , `Color`
- `Complex` (complex numbers)
- Small data containers

Records

What is a Record?

Records are reference types (or value types) designed for immutable data models with built-in value-based equality.

Record Classes (Reference Type)

```
// Positional syntax
public record Person(string FirstName, string LastName, int Age);

// Traditional syntax
public record Person
{
    public string FirstName { get; init; }
    public string LastName { get; init; }
    public int Age { get; init; }
}
```

Using Records

```
// Create instance
Person person1 = new Person("John", "Doe", 30);

// Records are immutable by default
// person1.Age = 31; // ERROR with init!

// Non-destructive mutation with 'with' expression
Person person2 = person1 with { Age = 31 };

Console.WriteLine(person1.Age); // 30
Console.WriteLine(person2.Age); // 31
```

Value-Based Equality

```
Person person1 = new Person("John", "Doe", 30);
Person person2 = new Person("John", "Doe", 30);

// Records use value equality
Console.WriteLine(person1 == person2); // True!

// Classes would use reference equality
// Would be False if these were classes
```

Record Structs (Value Type)

```
public record struct Point(double X, double Y);

// Or traditional syntax
public record struct Point
{
    public double X { get; set; }
    public double Y { get; set; }
}
```

Record Features

Automatic members:

1. **Primary constructor** (with positional syntax)
2. **Properties** for each parameter
3. **ToString()** override
4. **Equals()** and `GetHashCode()` override
5. **Copy constructor** (for `with` expressions)
6. **Deconstruction** support

```
public record Person(string FirstName, string LastName, int Age);

Person person = new Person("John", "Doe", 30);

// Deconstruction
var (first, last, age) = person;

// ToString
Console.WriteLine(person); // Person { FirstName = John, LastName = Doe, Age
= 30 }

// with expression
Person older = person with { Age = 31 };
```

Inheritance with Records

```
public record Person(string FirstName, string LastName);
public record Student(string FirstName, string LastName, string School)
    : Person(FirstName, LastName);

Student student = new Student("Jane", "Smith", "MIT");
```

When to Use Records

Use records when:

- Type is primarily for storing data
- You want immutability
- You need value-based equality
- You want built-in formatting
- Type models data transfer objects (DTOs)

Interfaces

What is an Interface?

An interface defines a **contract** - a set of members that implementing classes must provide. Interfaces contain no implementation (before C# 8.0 default interface members).

Declaring an Interface

```
public interface IShape
{
```

```
    // Properties
    double Area { get; }
    double Perimeter { get; }

    // Methods
    void Draw();
    void Resize(double factor);
}
```

Implementing an Interface

```
public class Circle : IShape
{
    public double Radius { get; set; }

    // Implement interface properties
    public double Area => Math.PI * Radius * Radius;
    public double Perimeter => 2 * Math.PI * Radius;

    // Implement interface methods
    public void Draw()
    {
        Console.WriteLine($"Drawing a circle with radius {Radius}");
    }

    public void Resize(double factor)
    {
        Radius *= factor;
    }
}
```

Multiple Interfaces

```
public interface IDrawable
{
    void Draw();
}

public interface IResizable
{
    void Resize(double factor);
}

// Class can implement multiple interfaces
public class Rectangle : IDrawable, IResizable
{
    public double Width { get; set; }
```

```
    public double Height { get; set; }

    public void Draw()
    {
        Console.WriteLine($"Drawing rectangle: {Width}x{Height}");
    }

    public void Resize(double factor)
    {
        Width *= factor;
        Height *= factor;
    }
}
```

Explicit Interface Implementation

```
public interface IControl
{
    void Paint();
}

public interface ISurface
{
    void Paint();
}

public class Sample : IControl, ISurface
{
    // Explicit implementation
    void IControl.Paint()
    {
        Console.WriteLine("IControl.Paint");
    }

    void ISurface.Paint()
    {
        Console.WriteLine("ISurface.Paint");
    }
}

// Usage
Sample obj = new Sample();
// obj.Paint(); // ERROR! Ambiguous

((IControl)obj).Paint();  // IControl.Paint
((ISurface)obj).Paint();  // ISurface.Paint
```

Default Interface Members (C# 8.0+)

```
public interface ILogger
{
    void Log(string message);

    // Default implementation
    void LogError(string message)
    {
        Log($"ERROR: {message}");
    }
}

public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }

    // LogError is inherited with default implementation
}
```

Interface Inheritance

```
public interface IShape
{
    double Area { get; }
}

public interface IColoredShape : IShape
{
    string Color { get; set; }
}

public class ColoredCircle : IColoredShape
{
    public double Radius { get; set; }
    public string Color { get; set; }

    public double Area => Math.PI * Radius * Radius;
}
```

When to Use Interfaces

Use interfaces when:

- You need to define a contract
- You want multiple inheritance behavior

- Different classes need common behavior
- You want to enable polymorphism
- Designing plugin architectures

Generics

What are Generics?

Generics allow you to create type-safe classes, methods, and interfaces that work with any data type while maintaining compile-time type safety.

Generic Classes

```
public class Box<T>
{
    private T item;

    public void Store(T item)
    {
        this.item = item;
    }

    public T Retrieve()
    {
        return item;
    }
}

// Usage
Box<int> intBox = new Box<int>();
intBox.Store(123);
int value = intBox.Retrieve();

Box<string> stringBox = new Box<string>();
stringBox.Store("Hello");
string text = stringBox.Retrieve();
```

Generic Methods

```
public class Helper
{
    public static T Max<T>(T a, T b) where T : IComparable<T>
    {
        return a.CompareTo(b) > 0 ? a : b;
    }
```

```
}

// Usage
int maxInt = Helper.Max(10, 20);            // 20
string maxString = Helper.Max("apple", "banana"); // "banana"
```

Multiple Type Parameters

```
public class Pair<TKey, TValue>
{
    public TKey Key { get; set; }
    public TValue Value { get; set; }

    public Pair(TKey key, TValue value)
    {
        Key = key;
        Value = value;
    }
}

// Usage
Pair<int, string> pair = new Pair<int, string>(1, "One");
```

Generic Constraints

```
// where T : struct - T must be a value type
public class ValueContainer<T> where T : struct
{
    public T Value { get; set; }
}

// where T : class - T must be a reference type
public class ReferenceContainer<T> where T : class
{
    public T Value { get; set; }
}

// where T : new() - T must have parameterless constructor
public class Factory<T> where T : new()
{
    public T Create()
    {
        return new T();
    }
}

// where T : BaseClass - T must derive from BaseClass
```

```
public class DerivedContainer<T> where T : Animal
{
    public T Animal { get; set; }
}

// where T : IInterface - T must implement IInterface
public class InterfaceContainer<T> where T : IComparable
{
    public T Value { get; set; }
}

// Multiple constraints
public class ComplexContainer<T> where T : class, IDisposable, new()
{
    public T Value { get; set; }
}
```

Generic Interfaces

```
public interface IRepository<T>
{
    void Add(T item);
    T GetById(int id);
    IEnumerable<T> GetAll();
    void Delete(int id);
}

public class ProductRepository : IRepository<Product>
{
    private List<Product> products = new List<Product>();

    public void Add(Product item)
    {
        products.Add(item);
    }

    public Product GetById(int id)
    {
        return products.FirstOrDefault(p => p.Id == id);
    }

    public IEnumerable<Product> GetAll()
    {
        return products;
    }

    public void Delete(int id)
    {
```

```
        products.RemoveAll(p => p.Id == id);
    }
}
```

Generic Delegates

```
// Generic delegate
public delegate T Operation<T>(T a, T b);

// Usage
Operation<int> add = (a, b) => a + b;
Operation<int> multiply = (a, b) => a * b;

int sum = add(5, 3);       // 8
int product = multiply(5, 3); // 15
```

Common Generic Collections

```
using System.Collections.Generic;

// List<T>
List<int> numbers = new List<int> { 1, 2, 3 };

// Dictionary<TKey, TValue>
Dictionary<string, int> ages = new Dictionary<string, int>
{
    { "John", 30 },
    { "Jane", 25 }
};

// HashSet<T>
HashSet<string> uniqueNames = new HashSet<string> { "John", "Jane" };

// Queue<T>
Queue<int> queue = new Queue<int>();
queue.Enqueue(1);
queue.Enqueue(2);

// Stack<T>
Stack<int> stack = new Stack<int>();
stack.Push(1);
stack.Push(2);
```

Benefits of Generics

1. **Type Safety** - Compile-time checking

2. **Performance** - No boxing/unboxing for value types
3. **Code Reuse** - Write once, use with many types
4. **Elimination of Casting** - No runtime type checks

Anonymous Types

What are Anonymous Types?

Anonymous types provide a convenient way to create temporary, read-only objects without explicitly defining a class.

Creating Anonymous Types

```
// Create anonymous type
var person = new
{
    FirstName = "John",
    LastName = "Doe",
    Age = 30
};

Console.WriteLine(person.FirstName); // John
Console.WriteLine(person.Age);       // 30

// Properties are read-only
// person.Age = 31; // ERROR!
```

Common Use Cases

1. LINQ Queries:

```
var products = new[]
{
    new { Name = "Laptop", Price = 1200, Category = "Electronics" },
    new { Name = "Mouse", Price = 25, Category = "Electronics" },
    new { Name = "Desk", Price = 300, Category = "Furniture" }
};

var query = products
    .Where(p => p.Price > 50)
    .Select(p => new
    {
        p.Name,
        PriceWithTax = p.Price * 1.1
    });
```

```
foreach (var item in query)
{
    Console.WriteLine($"{item.Name}: ${item.PriceWithTax:F2}");
}
```

2. Grouping Data:

```
var students = new[]
{
    new { Name = "John", Grade = "A", Subject = "Math" },
    new { Name = "Jane", Grade = "B", Subject = "Math" },
    new { Name = "Bob", Grade = "A", Subject = "Science" }
};

var grouped = students
    .GroupBy(s => s.Subject)
    .Select(g => new
    {
        Subject = g.Key,
        Count = g.Count(),
        AverageGrade = g.Average(s => s.Grade == "A" ? 4.0 : 3.0)
    });
```

3. Projection:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Phone { get; set; }
}

List<Customer> customers = GetCustomers();

// Project to anonymous type
var summary = customers.Select(c => new
{
    c.Id,
    c.Name,
    ContactInfo = $"{c.Email} / {c.Phone}"
});
```

Anonymous Type Arrays

```
var people = new[]
{
    new { Name = "John", Age = 30 },
    new { Name = "Jane", Age = 25 },
    new { Name = "Bob", Age = 35 }
};

// All elements must have the same structure
foreach (var person in people)
{
    Console.WriteLine($"{person.Name} is {person.Age} years old");
}
```

Equality Comparison

```
var person1 = new { FirstName = "John", LastName = "Doe" };
var person2 = new { FirstName = "John", LastName = "Doe" };

// Anonymous types override Equals
Console.WriteLine(person1.Equals(person2)); // True

// Same structure and values
Console.WriteLine(person1 == person2); // False (different instances)
```

Limitations

1. **Read-only** - Properties cannot be changed after creation
2. **Method scope** - Cannot be returned from methods (must use `object` or `dynamic`)
3. **Limited reusability** - Cannot be named or reused across methods
4. **No custom methods** - Cannot add methods to anonymous types

When to Use Anonymous Types

Good for:

- Temporary data structures in queries
- Grouping data within a single method
- Quick prototyping
- LINQ projections

Not good for:

- Passing data between methods
- Long-term storage
- Complex business logic

- When you need methods or behavior

Strings

What is a String?

In C#, `string` is a reference type that represents immutable sequences of Unicode characters.

String Declaration and Initialization

```
// Declaration
string str1;

// Initialization
string str2 = "Hello";
string str3 = "World";
string str4 = "";        // Empty string
string str5 = null;      // Null reference

// String literal
string quote = "He said, \"Hello!\"";
```

String Immutability

```
string original = "Hello";
string modified = original + " World"; // Creates NEW string

// original is unchanged
Console.WriteLine(original); // "Hello"
Console.WriteLine(modified); // "Hello World"
```

Verbatim Strings

```
// Regular string with escape sequences
string path1 = "C:\\Users\\John\\Documents\\file.txt";

// Verbatim string (@) - no escape sequences
string path2 = @"C:\Users\John\Documents\file.txt";

// Multi-line verbatim string
string multiline = @"Line 1
Line 2
Line 3";
```

String Interpolation

```
string name = "John";
int age = 30;

// Old way (concatenation)
string message1 = "My name is " + name + " and I'm " + age + " years old.";

// String interpolation ($)
string message2 = $"My name is {name} and I'm {age} years old.";

// With expressions
string message3 = $"Next year I'll be {age + 1} years old.";

// With formatting
double price = 19.99;
string formatted = $"Price: {price:C}"; // Price: $19.99
```

Common String Operations

```
string text = "Hello World";

// Length
int length = text.Length; // 11

// Access characters
char firstChar = text[0]; // 'H'
char lastChar = text[text.Length - 1]; // 'd'

// Substring
string sub = text.Substring(0, 5); // "Hello"
string sub2 = text.Substring(6);   // "World"

// ToUpper / ToLower
string upper = text.ToUpper(); // "HELLO WORLD"
string lower = text.ToLower
```